



Race-Track

ERIK BARTHOLOMÄUS – 135573

RENNSPIEL ADVANCED

ERIKAVEIRO/THREEJS-PROJECT-RACING

INTRODUCTION TO COMPUTER GRAPHICS – 2025/2026 – FINAL PROJECT

Content

- ▶ The Project
- ▶ Models
- ▶ Scene Graph
- ▶ Illumination
- ▶ Material & Textures
- ▶ Animation
- ▶ Interaction
- ▶ Development
- ▶ Use of AI
- ▶ Performance
- ▶ Conclusion

The Project

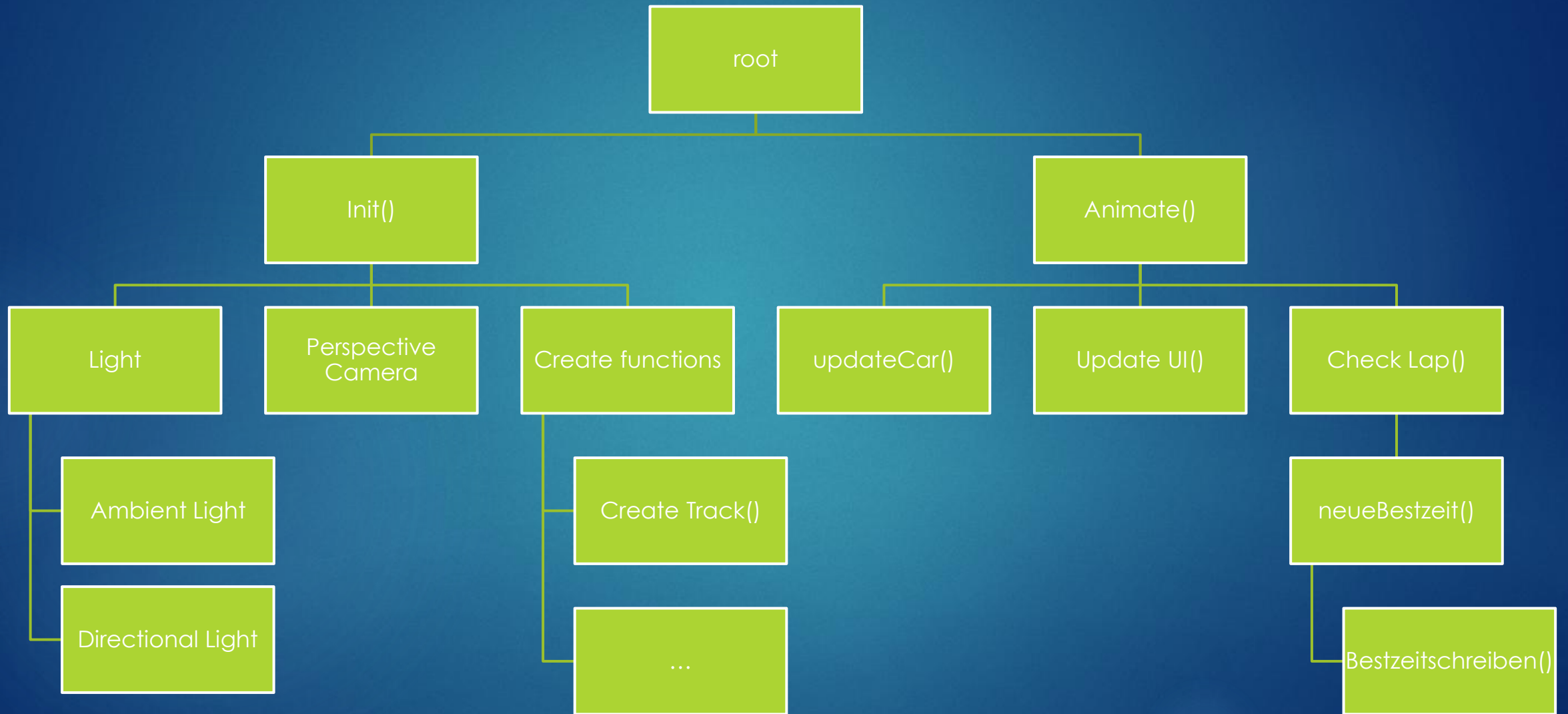
- ▶ Racing game
 - ▶ One track
 - ▶ Forest theme
 - ▶ Different variety of mechanics
- ▶ User controls Car model
 - ▶ WASD + Shift
- ▶ Basic Three.js (r184)



Models



Scene Graph



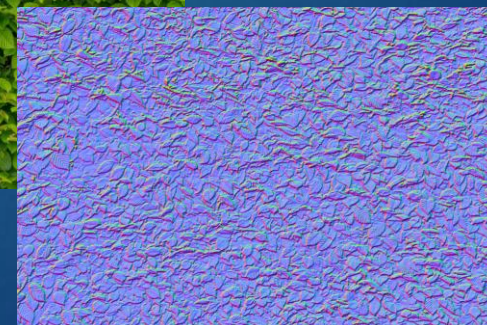
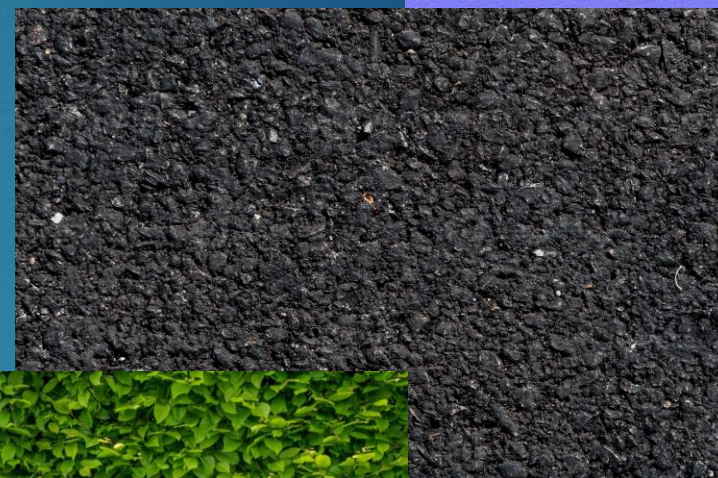
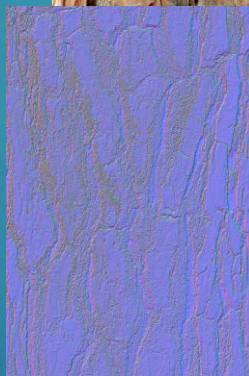
Illumination

- ▶ Ambient Light
 - ▶ Brightens up the game
- ▶ Directional Light
 - ▶ Used to simulate a sun in the distance
 - ▶ slightly red coloured
- ▶ Light is used to set the mood of the game!



Materials & Textures

- ▶ Standard Material
 - ▶ Most realistic
 - ▶ Highest quality of material
 - ▶ Easy to use for shadows
- ▶ Textures
 - ▶ Mapping
 - ▶ Normal mapping
 - ▶ Normal map generator
- ▶ Material & Texture are used to create realism!



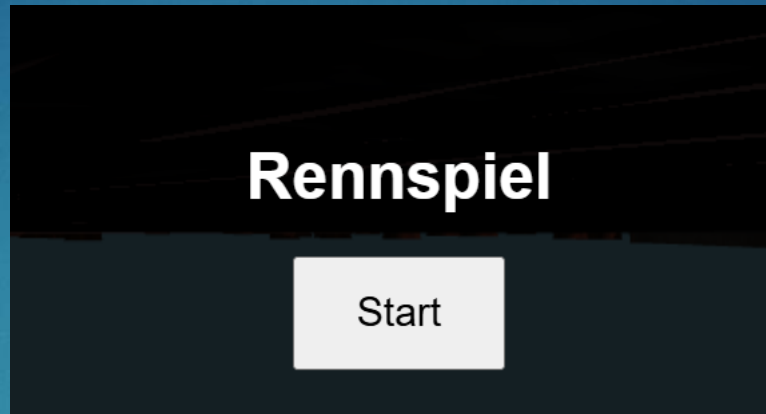
Animation

- ▶ UpdateCar()
 - ▶ Controls & Movement
- ▶ UpdateUI()
 - ▶ Time Measurement
 - ▶ Subtracts time since lap started from time passed
- ▶ CheckLap()
 - ▶ Lap completed?
 - ▶ Checks distance to starting point
- ▶ neueBestzeit()
 - ▶ New fastest lap?
- ▶ Animations are carried out in their own function rather than all inside the animate() function

```
function animate() {  
    requestAnimationFrame(animate);  
  
    const now = performance.now();  
    const delta = Math.min((now-lastTime)/16.666,2); // 60fps norm  
    lastTime = now;  
  
    if (started) {  
        updateCar(delta);  
        updateUI();  
        checkLap();  
    }  
  
    renderer.render(scene, camera);  
}
```


Interaction

- ▶ Perspective Camera
 - ▶ Realism
- ▶ Interaction
 - ▶ Controlling the car
 - ▶ WASD + Shift
 - ▶ Menu
 - ▶ To start the game
 - ▶ Controls UI



Steuerung

W: Vorwärts
A: Links
S: Rückwärts
D: Rechts
Shift: Drift

Development

- ▶ Visual Studio Code
 - ▶ Familiarity was already there
- ▶ Code Organization
 - ▶ Function/Object based
 - ▶ One file
 - ▶ Top Level: Init() & animate()
- ▶ Difficulties
 - ▶ Implementation of the Track
 - ▶ Shadows of the models
 - ▶ Performance

```
function init() {
  scene = new THREE.Scene();
  scene.background = new THREE.Color(0x87ceeb);

  camera = new THREE.PerspectiveCamera(75, innerWidth/innerHeight, 0.1, 1000);

  renderer = new THREE.WebGLRenderer({ antialias: true });
  renderer.setSize(innerWidth, innerHeight);
  renderer.shadowMap.enabled = true;
  renderer.shadowMap.type = THREE.PCFSoftShadowMap;
  document.body.appendChild(renderer.domElement);

  createGround();
  createTrack();
  Trees();
  ZielEinfahrt();
  WoodCabin();
  BestzeitanzeigeSetUp();
  loadCar();

  function animate() {
    requestAnimationFrame(animate);

    if (!car) {
      console.log("Auto noch nicht geladen...");
    }

    if (started) {
      updateCar();
      updateUI();
      checkLap();
      if (Bestzeit > Rundenzeit) {
        neueBestzeit(Rundenzeit);
      }
    }

    renderer.render(scene, camera);
  }
}
```

Use of AI

- ▶ Tool: ChatGPT
- ▶ Implementation of the custom Track with guiding rails
- ▶ Casting and receiving shadows on my models
- ▶ Making the drifting real
- ▶ Fixing performance issues

```
// Kontrollpunkte
const points = [
  new THREE.Vector3(0, 0, 0),
  new THREE.Vector3(0, 0, -80),
  new THREE.Vector3(-80, 0, -80),
  new THREE.Vector3(-80, 0, -60),
  new THREE.Vector3(-20, 0, -60),
  new THREE.Vector3(-20, 0, -40),
  new THREE.Vector3(-100, 0, -40),
  new THREE.Vector3(-100, 0, -100),
  new THREE.Vector3(0, 0, -100),
  new THREE.Vector3(40, 0, 0),
];
```

```
cabin.traverse((child)=>{
  if(child.isMesh){
    child.castShadow = true;
    child.receiveShadow = true;

    child.material.side = THREE.FrontSide;
  }
});
```

```
const forwardSpeed = velocity.dot(forward);
const sideSpeed = velocity.dot(right);

const driftFactor = 0.9; // weniger = mehr Drift

velocity.copy(forward.clone().multiplyScalar(forwardSpeed));

velocity.add(right.clone().multiplyScalar(sideSpeed * driftFactor));

else {
  // normaler Grip
  const forwardSpeed = velocity.dot(forward);
  velocity.copy(forward.clone().multiplyScalar(forwardSpeed));
```

```
const vertices = [];
const indices = [];
const uvs = [];

for (let i = 0; i < segments; i++) {

  const a = leftPoints[i];
  const b = rightPoints[i];
  const c = leftPoints[i + 1];
  const d = rightPoints[i + 1];

  const index = vertices.length / 3;

  vertices.push(a.x, a.y, a.z);
  vertices.push(b.x, b.y, b.z);
  vertices.push(c.x, c.y, c.z);
  vertices.push(d.x, d.y, d.z);

  const v = i / segments;
  const vNext = (i + 1) / segments;

  uvs.push(0, v);
  uvs.push(1, v);
  uvs.push(0, vNext);
  uvs.push(1, vNext);

  indices.push(index, index + 1, index + 2);
  indices.push(index + 2, index + 1, index + 3);
}
```


Performance

- ▶ Track
 - ▶ Amount of segments the track is made of
- ▶ Nearest Point
 - ▶ How accurate the „nearest Point“ is calculated
- ▶ Trees
 - ▶ How large the area is in which trees appear
- ▶ Instanced Meshes for Trees & guardrails
- ▶ Model files needed!
- ▶ Only PC (WASD +Shift)
- ▶ What would I do differently?
 - ▶ Program organization first!

```
// Strecke  
function createTrack() {  
  
    trackWidth = 6;  
    const segments = 200;  
}
```

```
//Kollisionspunkt finden  
function getNearestPointOnTrack(position) {  
  
    const divisions = 2000;  
    const areaSize = 600;  
}
```

```
const trunks = new THREE.InstancedMesh(trunkGeo, trunkMat, maxTrees);
```

```
// Strecke  
function createTrack() {  
  
    trackWidth = 6;  
    const segments = 20000;  
}
```



Demo

References

- ▶ Models
 - ▶ [Download Free 3D Models - Royalty Free – Sketchfab](#)
 - ▶ [Download Free 3D Models - Royalty Free – Sketchfab](#)
- ▶ Textures
 - ▶ [Free Normal Map Generator Online | Free 3D Texture Tool](#)
- ▶ AI
 - ▶ [ChatGPT](#)
- ▶ Books/sites
 - ▶ [three.js docs](#)
 - ▶ [three.js examples](#)
- ▶ Tutorials
 - ▶ [Bring your ideas to life with Base44](#)